Design and Implementation of Arm-CCA compatible Unikernel

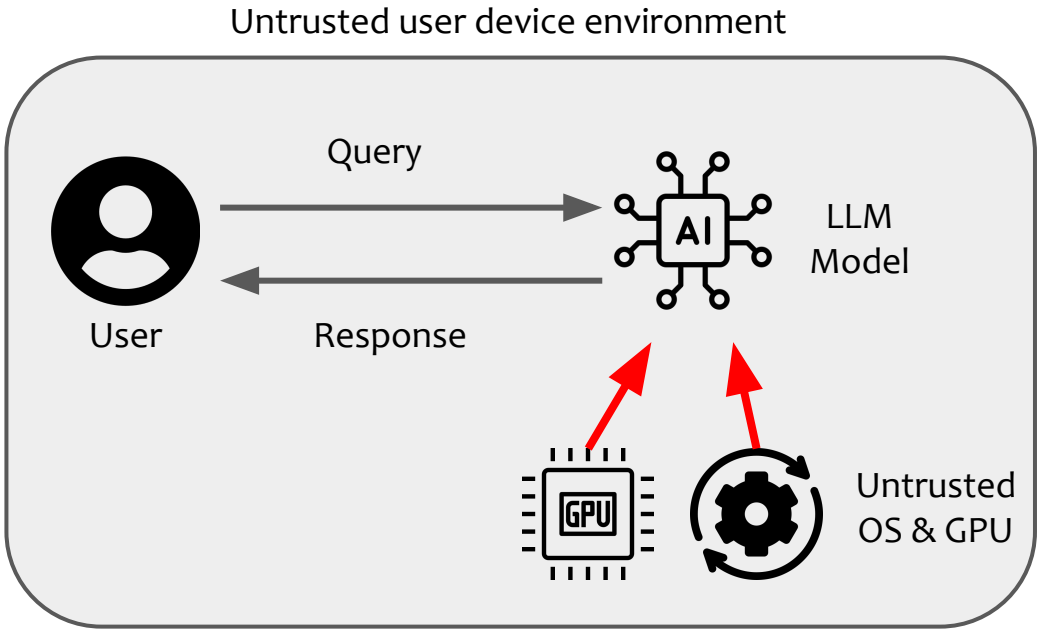
Johanna Latzel
Advisor: Patrick Sabanic
Examiner: Prof. Dr. Pramod Bhatothia
Systems Research Group
https://dse.cit.tum.de/

04.11.2024 – 04.05.2025

On-device LLM Inference in Untrusted Environments

Security goals for LLM inference

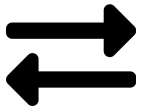
- User data security and privacy
- Model security



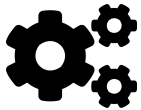
How to protect data and model from the untrusted host?

Challenges of Secure LLM Inference

Security of different states of data



data-in-transit



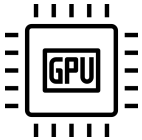
data-in-use

High performance requirements

limited device ressources

High-privilege attackers

hypervisor / OS

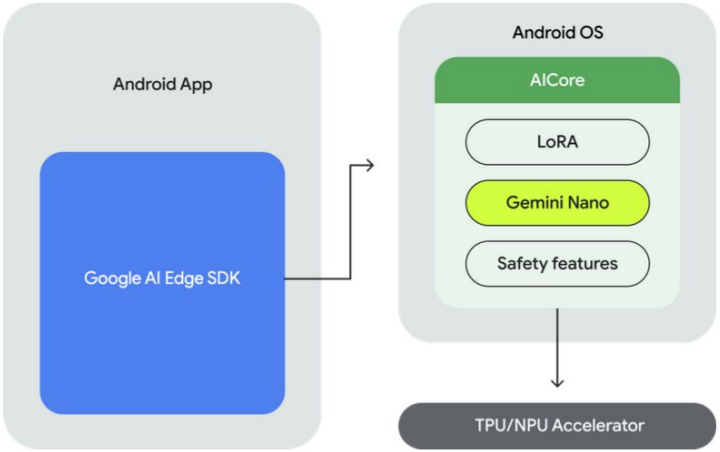


DMA-devices

How can we address these challenges?

Research Gap

- Android AICore
  - Isolates LLM and user data
  - Compromised OS ⟶ Compromised data and model
  - Security is not attestable



AICore Architecture: https://developer.android.com/ai/gemini-nano

Outline

- ~~Motivation~~
- Overview
- Design & Implementation
- Evaluation
- Conclusion

Problem statement

How can we design a system for **secure** and **performant** LLM inference in hostile environments?

Proposal

A Unikernel as an ArmCCA Confidential VM for Secure LLM Inference

System design goals:

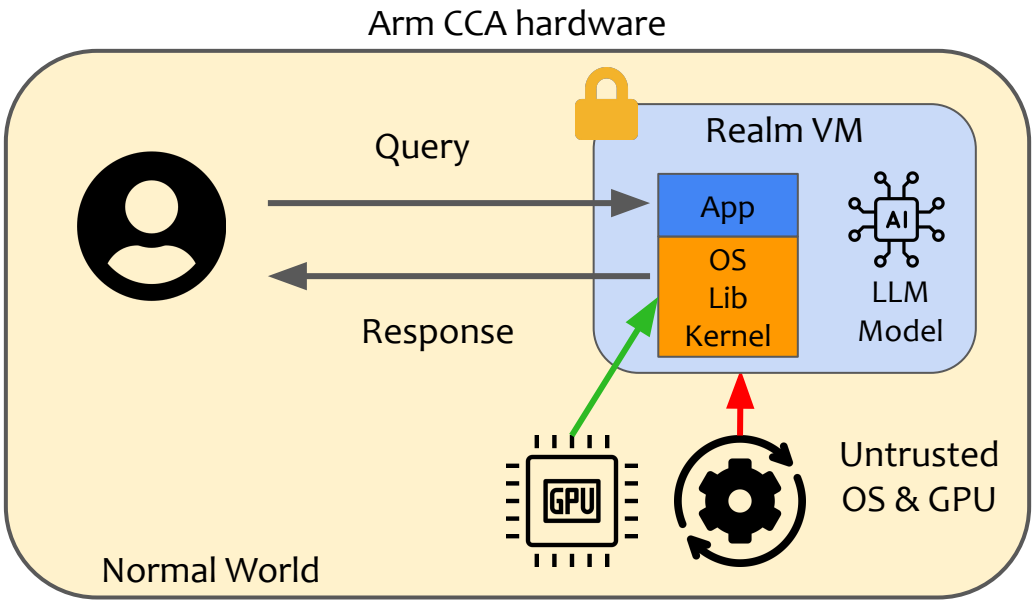
- **Security**
- **Performance**
- **Adaptability**

Arm CCA

Arm **CCA:**

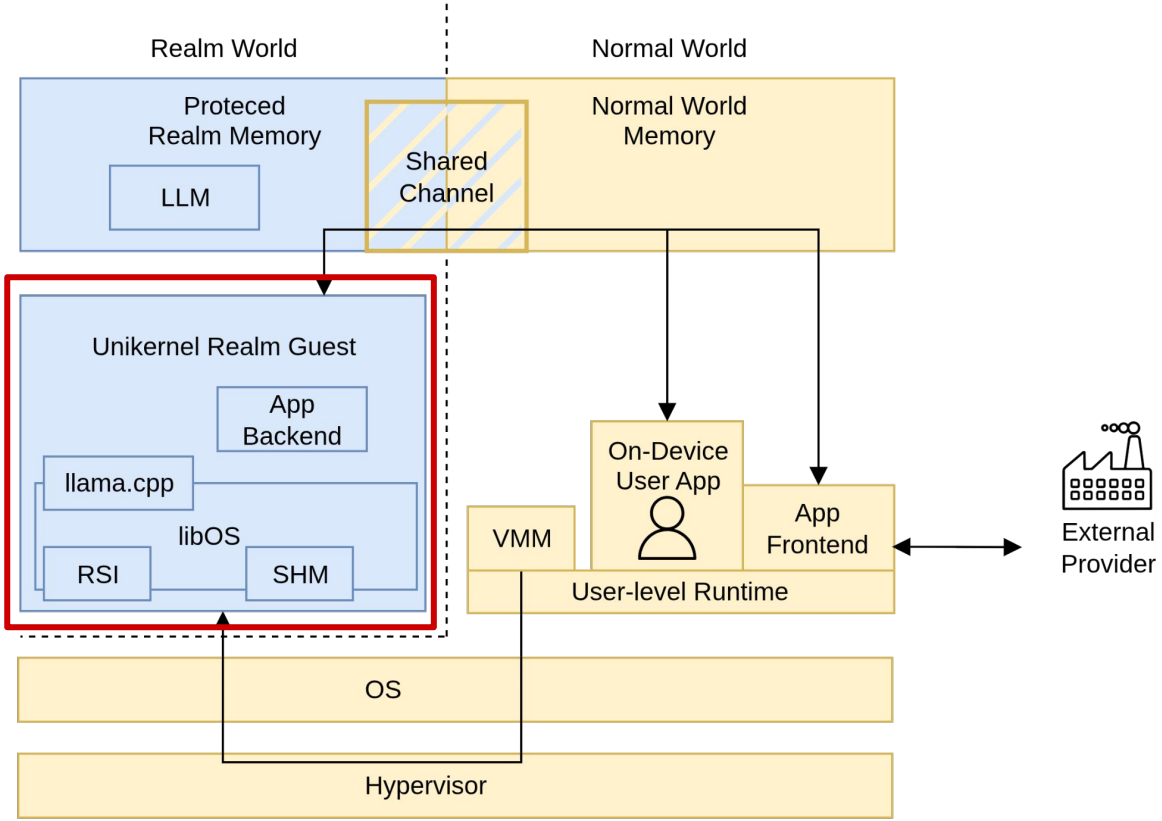
Confidential **C**ompute on **A**rm

- Hardware-based TEE
- Open-source software stack



Arm CCA protects the model and data while in use.

Overview

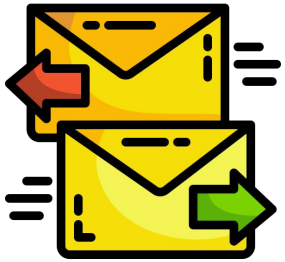

Realm World
Normal World
Proteced
Realm Memory
LLM
Shared
Channel
Normal World
Memory
Unikernel Realm Guest
App
Backend
llama.cpp
libOS
RSI
SHM
On-Device
User App
VMM
App
Frontend
User-level Runtime
External
Provider
OS
Hypervisor

Outline

- ~~Motivation~~
- ~~Overview~~
- Design & Implementation
- Evaluation
- Conclusion

Challenges

Lightweight OS

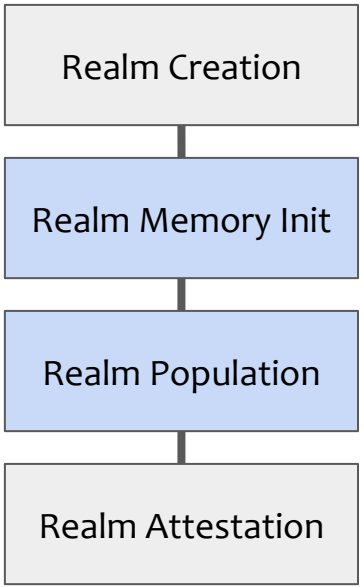


Efficient Communication

Challenge #1: Lightweight Arm CCA Realm Guest

- Confidential computation creates a performance overhead

- Increasing size of guest image → Increasing overhead

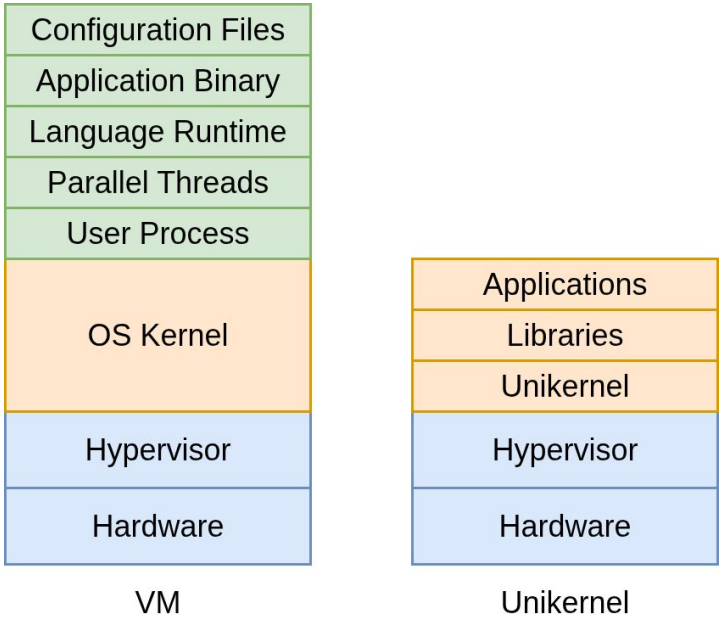


How to ensure low overheads?

Solution #1: Unikernel Architecture

- Single address space

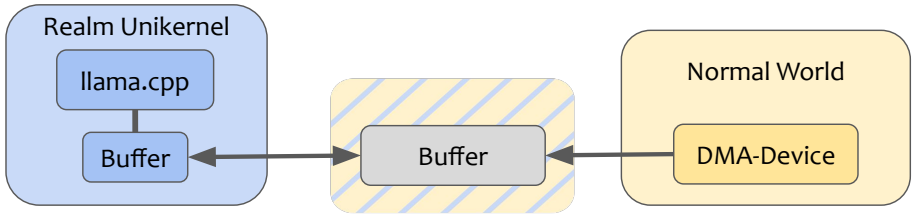
- Modular library operating system



The unikernel architecture allows building light specialized VMs.

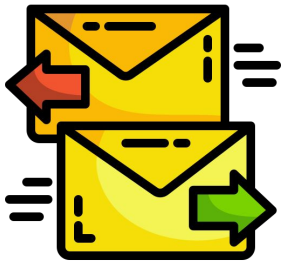
Solution #1: Unikernel Modifications

- RSI: Realm Service Interface
  - Configuration
  - Sharing and protection of memory
  - Generation of attestation token
- Realm Boot Process
  - Realm memory protection
  - Basic device memory sharing
- Shared Device Access
  - Proof-of-concept VirtIO 9pfs
  - Shared memory allocation and bounce buffering



Challenges

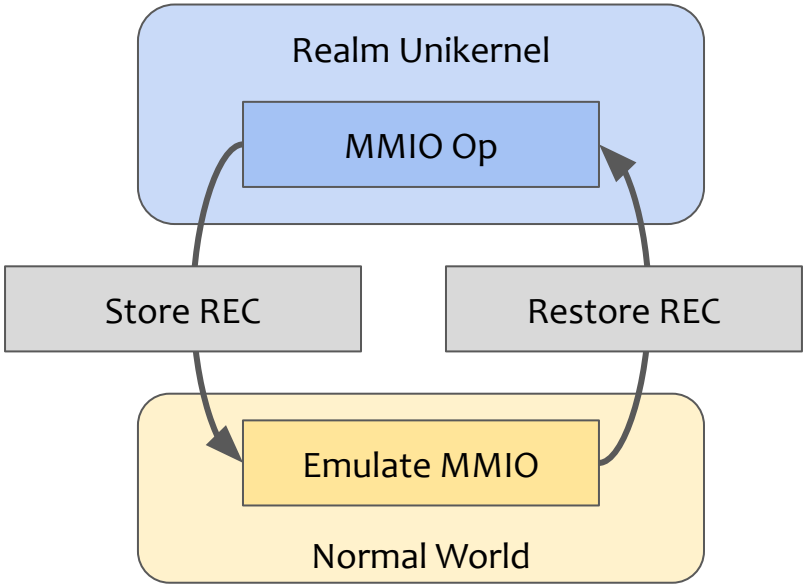
Lightweight OS



Efficient Communication

Challenge #2: Efficient Communication

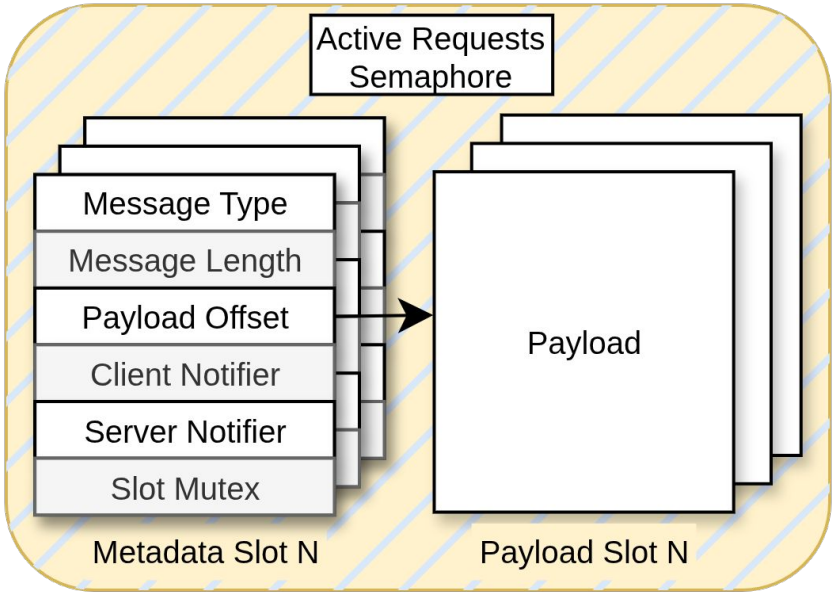
- VirtIO devices cause MMIO exits
- VirtIO devices require bounce buffering
- VirtIO adds a paravirtualized device layer



How to communicate with the Normal world efficiently?

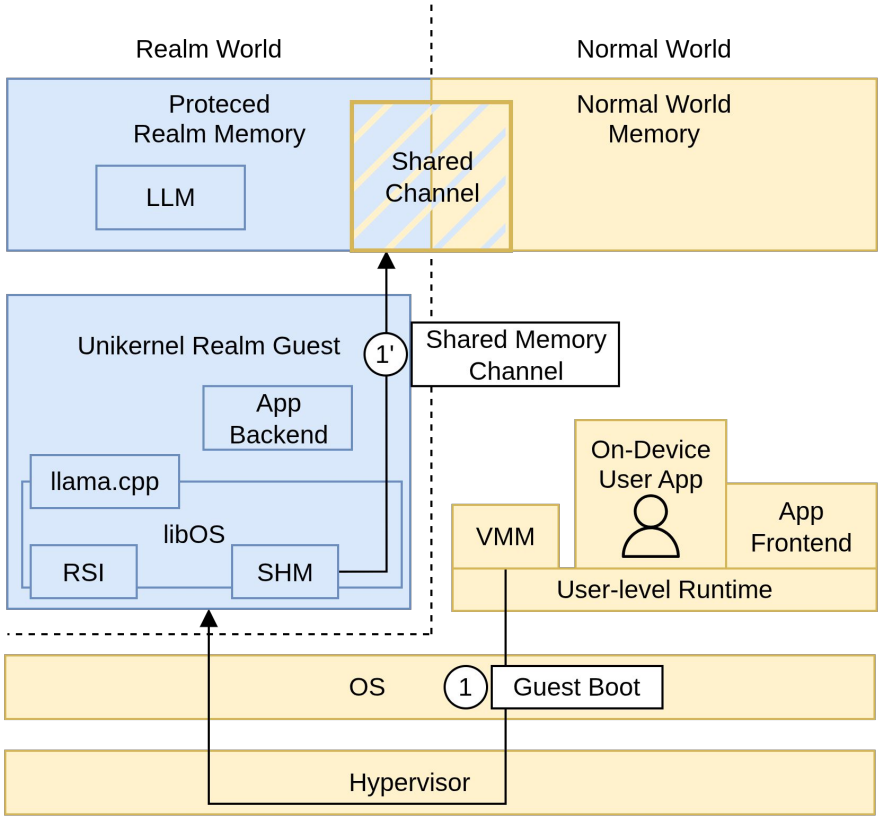
Solution #2: Shared Memory Channel

- Pre-shared static memory region
- Custom communication protocol
- Data can be processed directly from the payload region

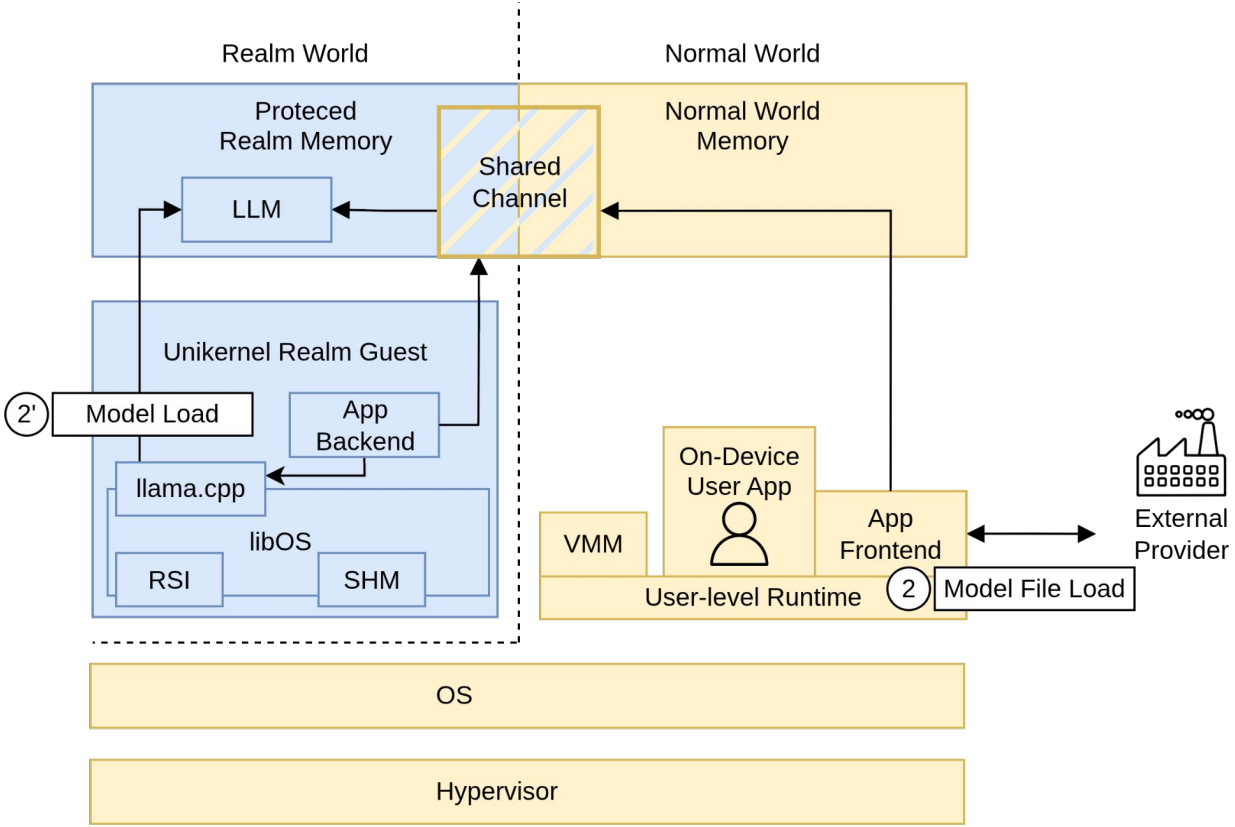


Design a slot-based shared memory communication channel.

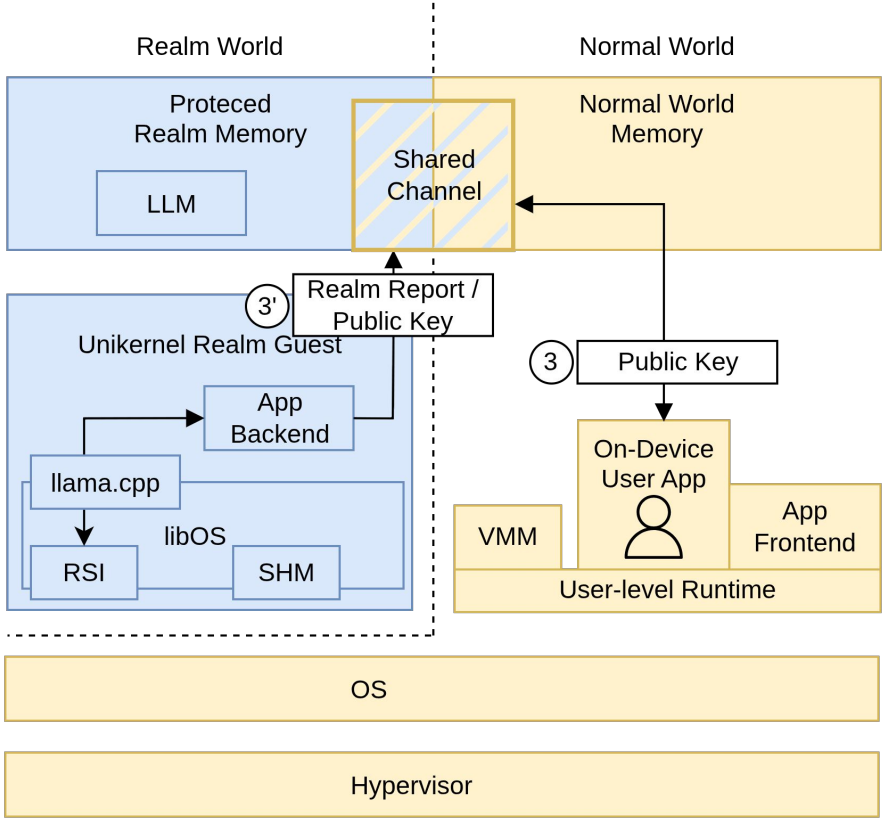
Workflow - Guest Startup



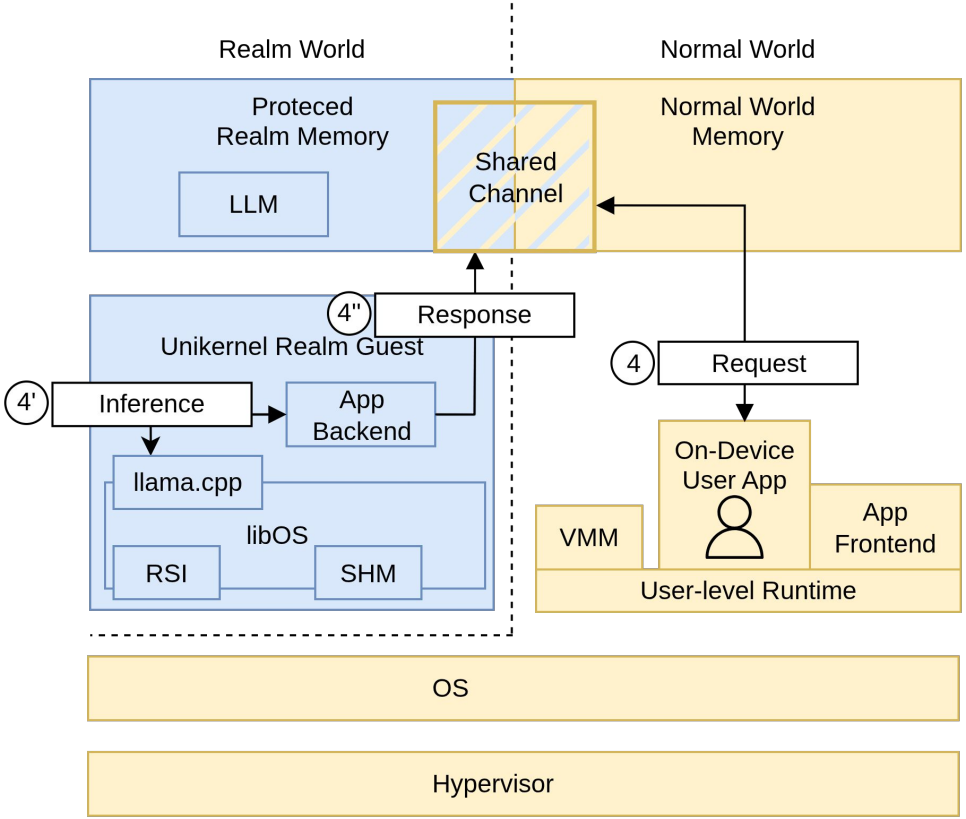
Workflow - Model Load



Workflow - Secure Channel



Workflow - Inference Service



Implementation

- Modification of Unikraft unikernel
- Hypervisor: KVM
- VMM:

VMM	Supported Communication	Runtime Environment
Kvmtool	Shared Memory, limited VirtIO	QEMU-RME, FVP, OpenCCA
QEMU	Shared Memory	QEMU-RME

- CPU-based llama.cpp inference engine (LROS)

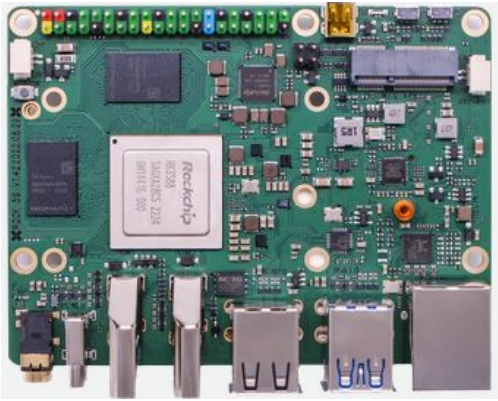
Outline

- ~~Motivation~~
- ~~Overview~~
- ~~Design & Implementation~~
- Evaluation
- Conclusion

Experiment Setup

- ArmCCA: OpenCCA Framework

- Hardware: Radxa Rock 5B
  - Rockchip RK3588 SoC
  - 4 GB (OpenCCA Restriction)
  - 4x Cortex-A55 (OpenCCA Restriction)



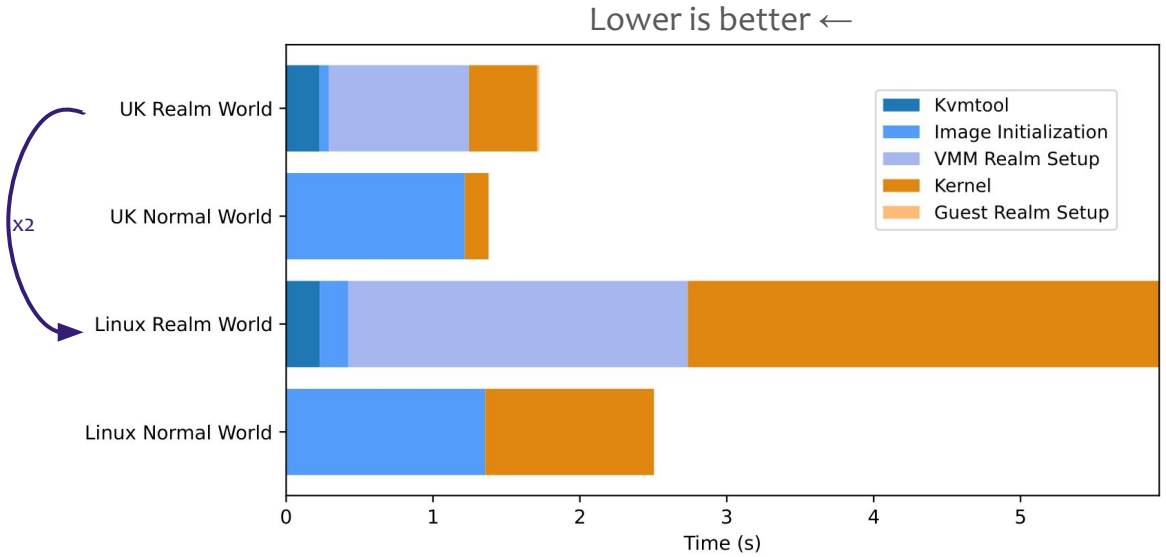
Linux Realm Overhead versus Unikraft Realm Overhead

Boot Phase

- Hypervisor Initialization
- Kernel Initialization

Image Size

- Unikernel: 9MB
- Linux VM: 42.4MB



Unikraft Realm guest significantly lighter than Linux Realm guest.

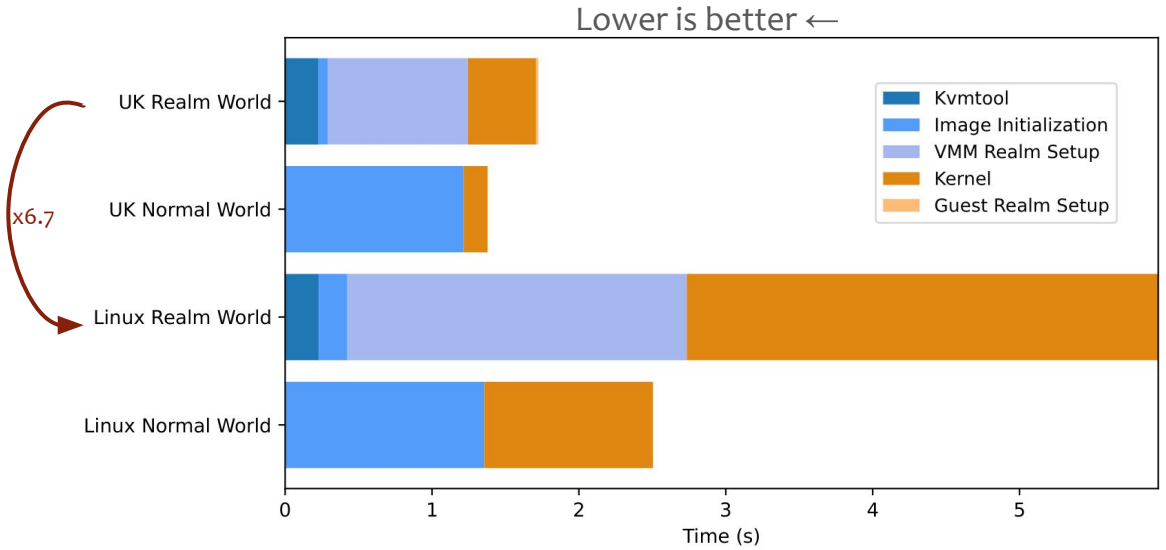
Linux Realm Overhead versus Unikraft Realm Overhead

Boot Phase

- Hypervisor Initialization
- Kernel Initialization

Image Size

- Unikernel: 9MB
- Linux VM: 42.4MB



Unikraft Realm guest significantly lighter than Linux Realm guest.

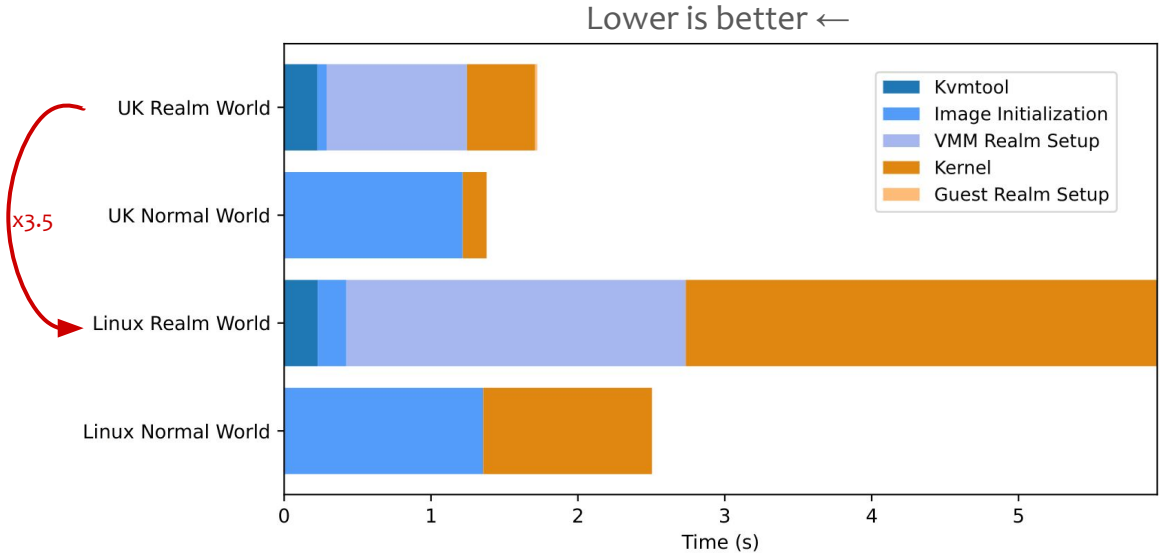
Linux Realm Overhead versus Unikraft Realm Overhead

Boot Phase

- Hypervisor Initialization
- Kernel Initialization

Image Size

- Unikernel: 9MB
- Linux VM: 42.4MB

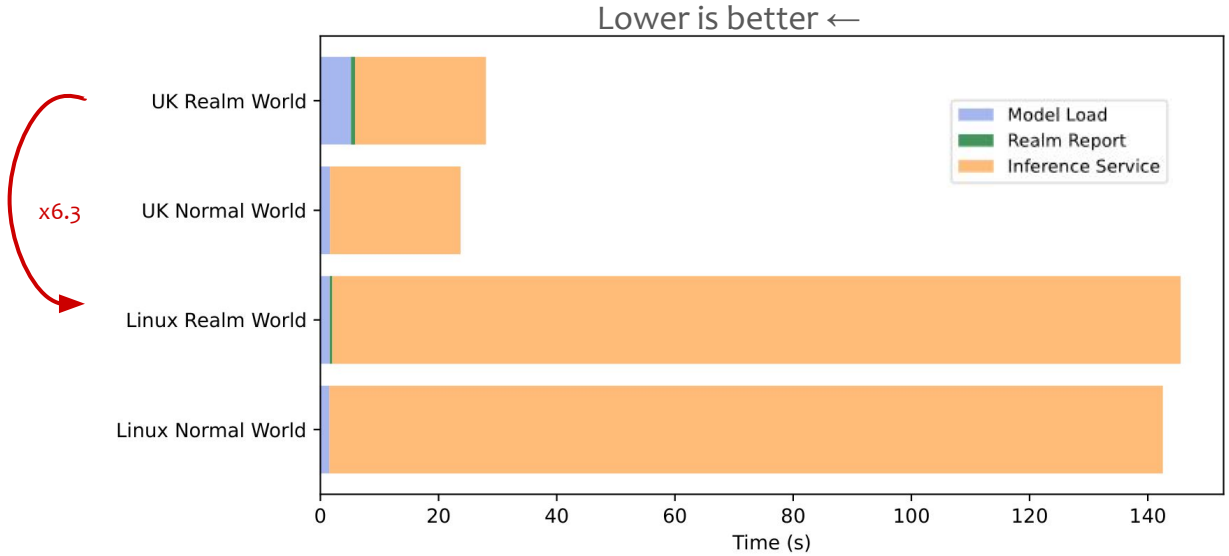


Unikraft Realm guest significantly lighter than Linux Realm guest.

Linux Realm Overhead versus Unikraft Realm Overhead

Application Phase

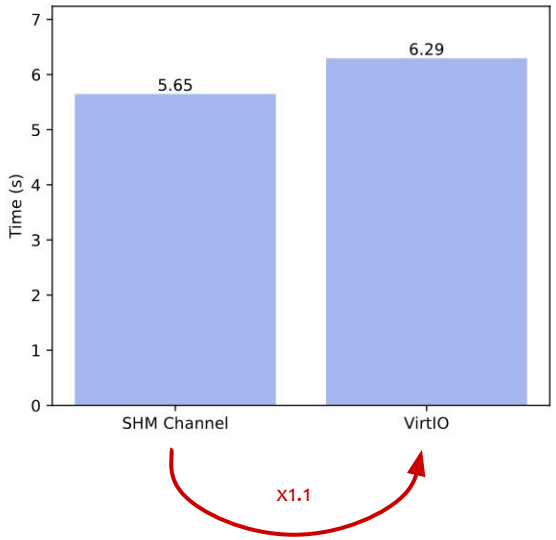
- Model Load
- Realm Report
- Inference



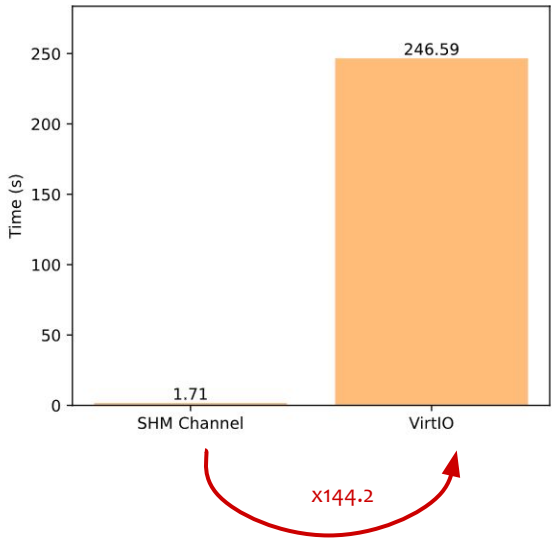
Further investigation required. Unikernel advantage mostly expected during boot.

VirtIO vs Shared Memory Communication

llama.cpp Model Load



Sequential File Load



VirtIO is more adaptable and flexible. Shared memory is much faster.

Conclusion

Key Contributions:

- Basis for Unikraft as an Arm CCA Realm guest
- Shared memory channel between Normal and Realm World
- Design of an end-to-end secure inference service
- First evaluation of unikernel performance in ArmCCA

Future Work:

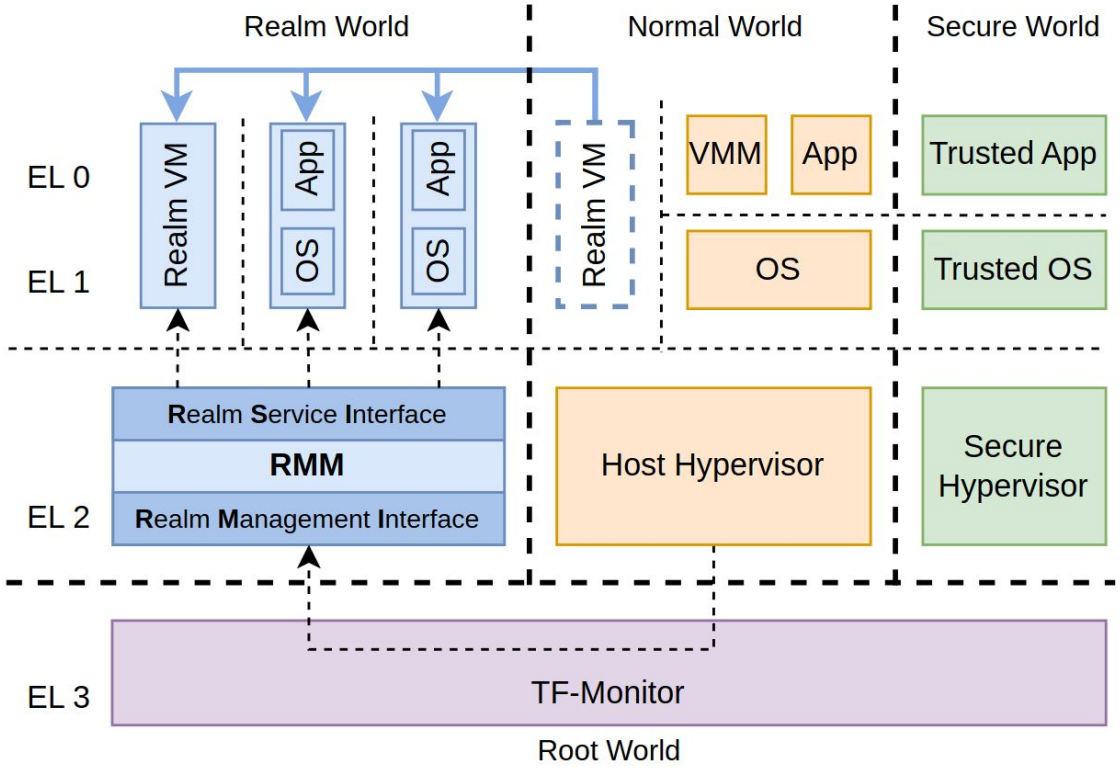
- Shared Device Access
- Secure Device Access: RME-DA or custom change to TCB

Backup

Arm CCA Architecture

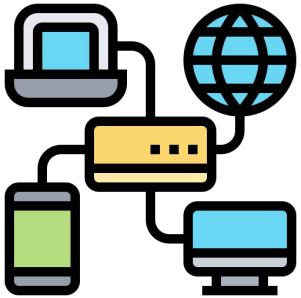
Arm CCA additions:

- Realm World
- Root World

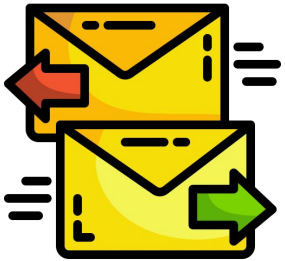


Challenges

Lightweight OS



Device Access



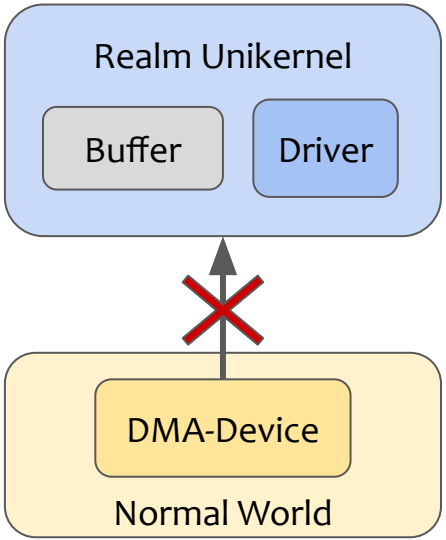
Efficient Communication

Secure Communication

Challenge #2: Device Access

- Secure device access
  - ArmCCA secure device assignment
  - Custom changes to the TCB

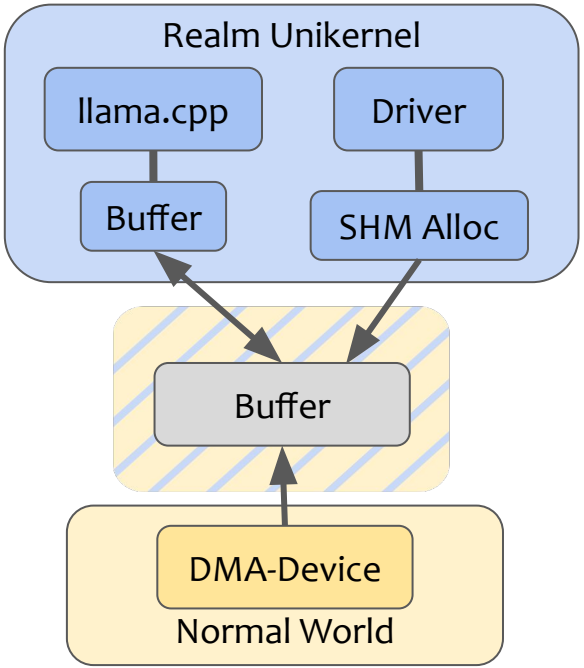
- Insecure device access
  - Normal world devices cannot access Realm
  - Unikraft does not implement bounce buffers



How to enable device access in the isolated Realm?

Solution #2: Shared VirtIO

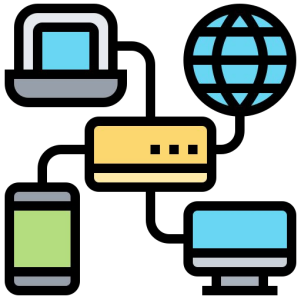
- Dynamic sharing causes VM exits and delay
- Device allocates buffer with shared memory allocator
- External applications still allocate protected buffers



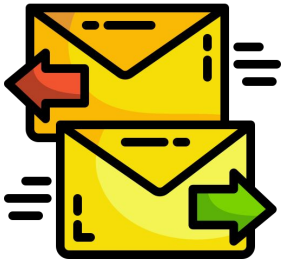
Create a custom allocator to a reserved and shared static memory region.

Challenges

Lightweight OS



Device Access

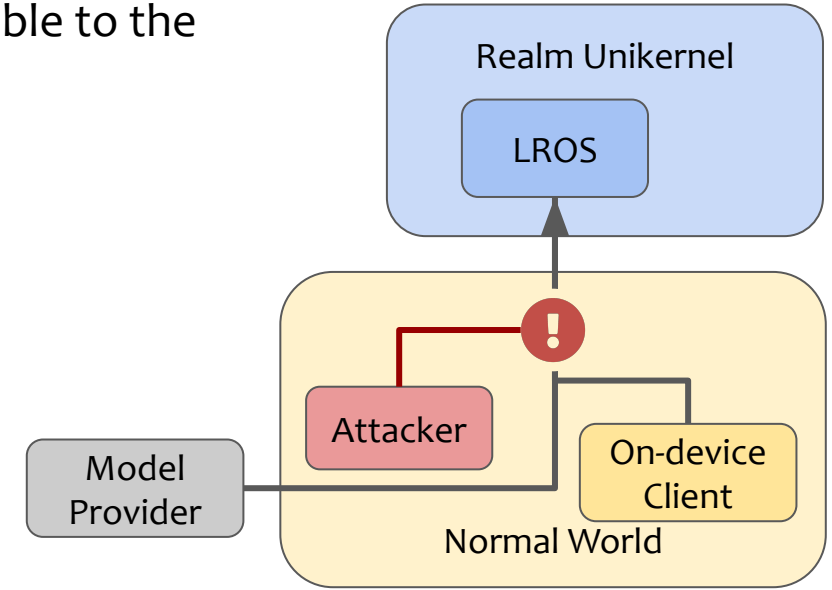


Efficient Communication

Secure Communication

Challenge #4: Secure Communication

- VirtIO and shared memory are vulnerable to the Normal world

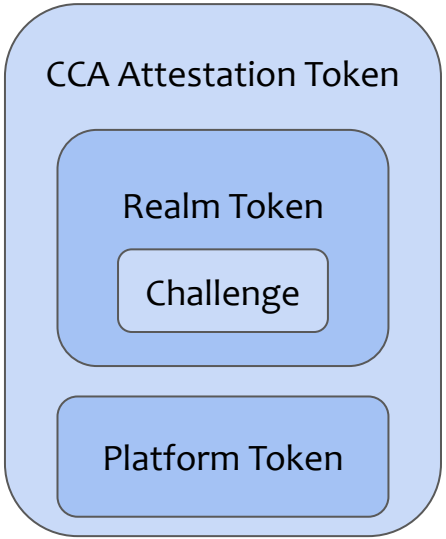


How can we ensure secure communication for sensitive data?

Challenge #4: Key Exchange during Attestation

- Realm attestation report guarantees

- DH public key as realm attestation challenge



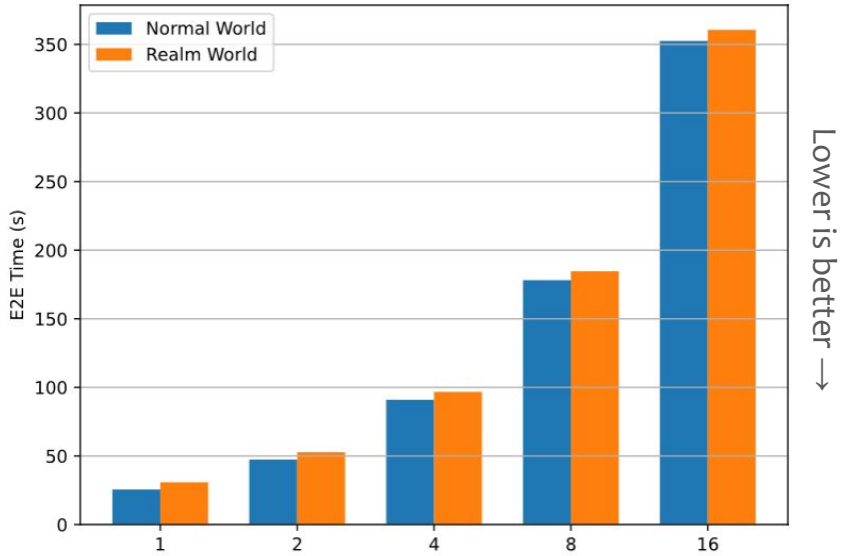
Bind the public key to the attestation token.

Unikraft Realm Overhead

Overhead with multiple clients

1 Client: 20% Realm overhead

→ 16 Clients: 2% Realm overhead

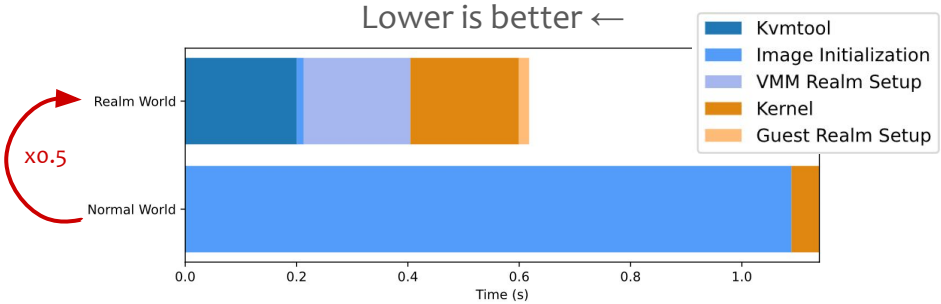


Most of the overhead is caused by the initialization and first page accesses.

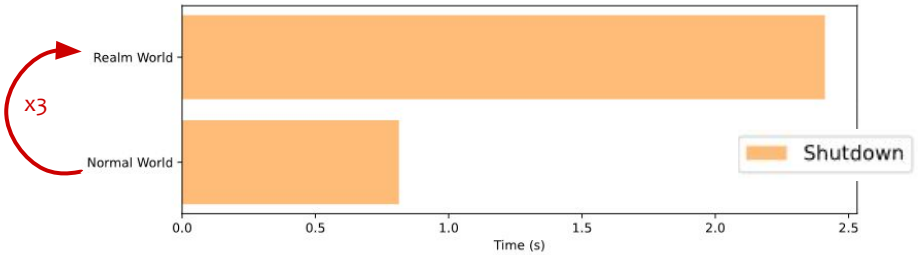
Unikraft Realm Overhead

Boot Phase

- Kvmtool Initialization
- Kernel Initialization



Shutdown Phase



The Realm overhead only becomes significant during page backing.

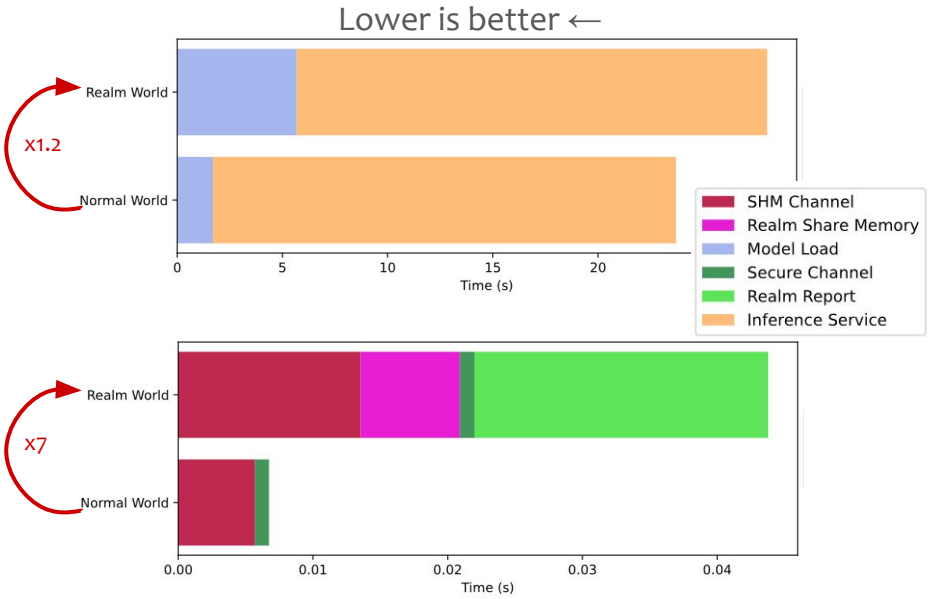
Unikraft Realm Overhead

Application Phase

- Model Load
- Inference

Zoom: Secure Communication

- Shared Memory Channel setup
- Attestation and Key Exchange



Secure Communication generates a comparatively minimal overhead.

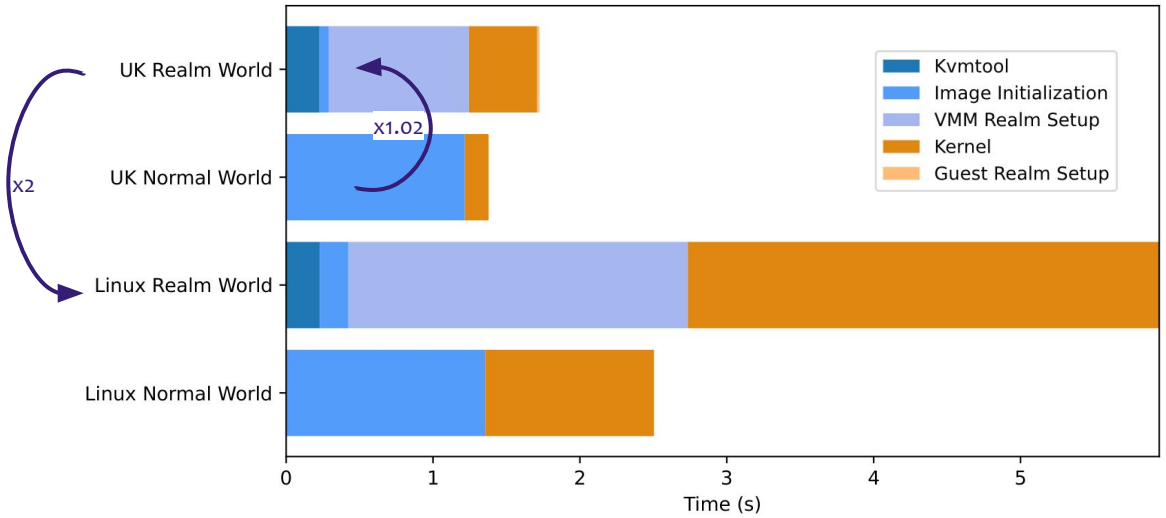
Linux Realm Overhead versus Unikraft Realm Overhead

Boot Phase

- Hypervisor Initialization
- Kernel Initialization

Image Size

- Unikernel: 9MB
- Linux VM: 42.4MB



Unikraft Realm guest significantly lighter than Linux Realm guest.

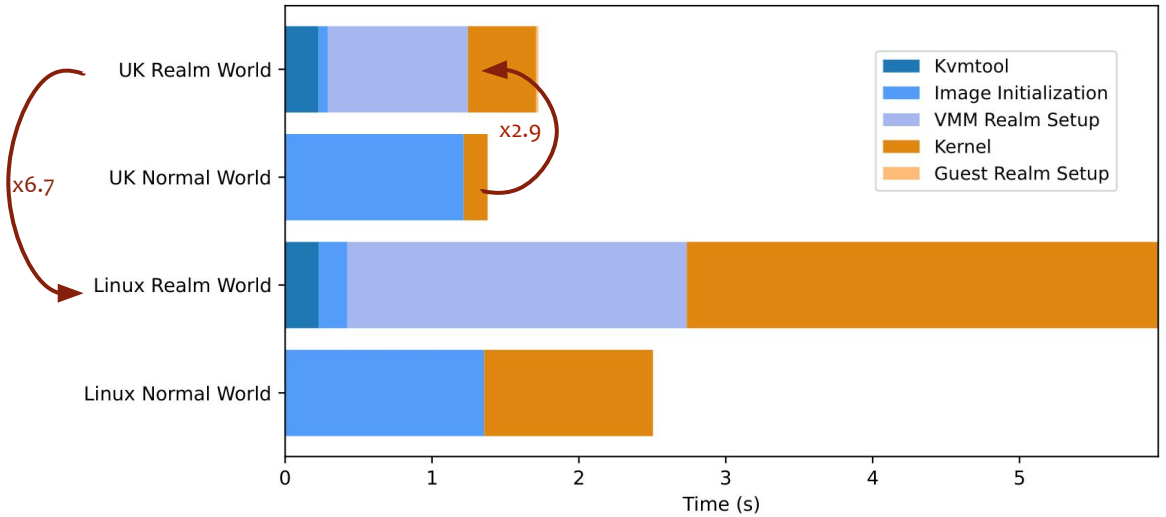
Linux Realm Overhead versus Unikraft Realm Overhead

Boot Phase

- Hypervisor Initialization
- Kernel Initialization

Image Size

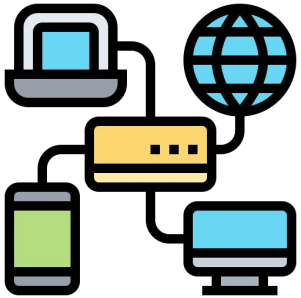
- Unikernel: 9MB
- Linux VM: 42.4MB



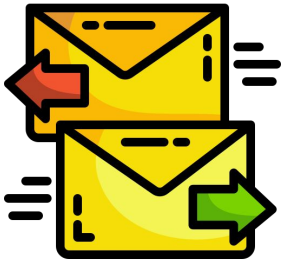
Unikraft Realm guest significantly lighter than Linux Realm guest.

Challenges

Lightweight OS



Device Access



Efficient Communication

Secure Communication